

THE COMPANION

Mathematical Appendices, References, and Glossary

The reference layer for the three-volume LLM systems series

Book 1 • Book 2 • Book 3

Carmel Prosper Sagbo

Research Associate & ML Infrastructure Engineer
CyLab-Africa / Upanzi Network
Carnegie Mellon University Africa, Kigali

June 2026

For internal study and reference

Contents

Purpose of This Volume	v
I Mathematical Appendices	1
1 Attention and Softmax	3
1.1 Scaled Dot-Product Attention	3
1.1.1 Why the dot product	3
1.1.2 Why divide by $\sqrt{d_k}$	3
1.2 Softmax and Its Gradient	3
1.2.1 Numerical stability	4
1.2.2 The Jacobian	4
1.2.3 Temperature	4
2 Positional Encoding and RoPE	5
2.1 The Rotation	5
2.2 Why the Score Becomes Relative	5
3 LoRA: Low-Rank Adaptation	7
3.1 The Factorization	7
3.2 The Parameter Saving	7
4 Alignment Objectives	9
4.1 PPO for RLHF	9
4.2 The Reward Model	9
4.3 DPO: Skipping the Reward Model	9
5 Distributed Communication	11
5.1 All-Reduce	11
5.2 FSDP Communication	11
5.3 Expert Parallelism (MoE)	11
II Annotated Bibliography	13
6 Foundational References	15
6.1 Transformers and Attention	15
6.2 Positional Encoding	15
6.3 Architectures Beyond the Transformer	15

6.4	Scaling and Training	16
6.5	Fine-Tuning and Alignment	16
6.6	Mixture of Experts	16
6.7	Inference Systems and Serving	16
6.8	Quantization	17
6.9	Memory-Saving Training	17
6.10	Retrieval and Reasoning	17
III	Glossary	19
7	Cross-Book Glossary	21
	How the Companion Maps to the Trilogy	25

Purpose of This Volume

The three main books were written to be read — diagram first, intuition first, mathematics kept light so the ideas stay visible. That choice was deliberate, but it leaves three things for a reader who wants more rigor or who is preparing the work for publication.

This companion supplies them, in one place, so the main text stays clean:

1. **Mathematical appendices** (Part I): the derivations the main text states as results — scaled dot-product attention, softmax and its gradient, RoPE, LoRA's factorization, the PPO and DPO objectives, and the communication cost of FSDP. Each appendix names where in the trilogy the result is used.
2. **Annotated bibliography** (Part II): the foundational papers behind each major concept, grouped by topic, with a one-line note on why each matters. The main books teach the ideas; this is where they are attributed.
3. **Glossary** (Part III): a single alphabetical reference for the terms that recur across all three books, each with a pointer to the book and chapter where it is introduced.

Intuition You do not need this volume to follow the trilogy. Read it when you want to see the algebra behind an idea, find the original paper to go deeper, or look up a term quickly. It is a reference shelf, not a chapter.

A note on cross-references: throughout, a marker like [B2: Ch. 9] points to Book 2, Chapter 9. **B1**, **B2**, **B3** denote the three books.

Part I

Mathematical Appendices

Attention and Softmax

1.1 Scaled Dot-Product Attention

[B1: Ch. 1] The attention operation maps a set of queries against keys and values. Stack the per-token queries, keys, and values into matrices $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{n \times d_k}$, $V \in \mathbb{R}^{n \times d_v}$, where n is the sequence length. Then

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

1.1.1 Why the dot product

For a single query $\mathbf{q}$ and key $\mathbf{k}$, the score $\mathbf{q} \cdot \mathbf{k} = \|\mathbf{q}\| \|\mathbf{k}\| \cos \theta$ is large when the two vectors point in a similar direction. Attention is therefore a learned similarity: the model shapes Q and K through W_Q, W_K so that tokens which should attend to each other produce large dot products.

1.1.2 Why divide by $\sqrt{d_k}$

Assume the entries of $\mathbf{q}$ and $\mathbf{k}$ are independent with zero mean and unit variance. Then their dot product

$$\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$$

has mean 0 and variance d_k (a sum of d_k independent unit-variance terms). Its standard deviation grows like $\sqrt{d_k}$. For large d_k the raw scores are large in magnitude, which pushes softmax into a region where one entry dominates and its gradient nearly vanishes (next section). Dividing by $\sqrt{d_k}$ rescales the scores back to unit variance, keeping softmax in a well-behaved range.

This single $\sqrt{d_k}$ is why attention trains stably at large head dimensions. Without it, deeper or wider models would see softmax saturate and gradients die — the vanishing-gradient failure mode of [B3: Ch. 6].

1.2 Softmax and Its Gradient

[B1: Ch. 5] For a logit vector $\mathbf{z} \in \mathbb{R}^V$, softmax produces a probability distribution:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}.$$

1.2.1 Numerical stability

Exponentials overflow for large z_i . The standard fix subtracts the maximum, which leaves the result unchanged:

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \quad m = \max_j z_j.$$

This is exactly the shift the *online softmax* of Flash Attention maintains incrementally [B2: Ch. 9]: a running maximum m updated tile by tile, with the accumulated sum rescaled by $e^{m_{\text{old}} - m_{\text{new}}}$ whenever a larger maximum appears.

1.2.2 The Jacobian

Differentiating p_i with respect to z_j gives the softmax Jacobian:

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j),$$

where δ_{ij} is 1 if $i = j$ and 0 otherwise. Two consequences: when one $p_i \rightarrow 1$ (a saturated, low-temperature distribution) the derivatives $p_i(1 - p_i) \rightarrow 0$, so learning slows — the motivation for the $\sqrt{d_k}$ scaling above and for temperature control [B1: Ch. 5]. The off-diagonal $-p_i p_j$ encodes that raising one probability must lower the others, since they sum to one.

1.2.3 Temperature

Temperature T rescales logits before softmax:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}.$$

As $T \rightarrow 0$ the distribution concentrates on $\arg \max_i z_i$ (greedy); as $T \rightarrow \infty$ it tends to uniform. The entropy $H = -\sum_i p_i \log p_i$ increases monotonically with T , which is the “effective vocabulary” 2^H tracked in [B1: Ch. 5].

Positional Encoding and RoPE

[B3: Ch. 1] RoPE encodes position by *rotating* query and key vectors by an angle proportional to position, so that attention scores depend only on relative position.

2.1 The Rotation

Split a d -dimensional vector into $d/2$ pairs of coordinates. For position m , rotate the k -th pair by angle $m\theta_k$, where $\theta_k = 10000^{-2k/d}$ (the same frequency schedule as sinusoidal encodings). For one pair (x_1, x_2) at position m :

$$\begin{pmatrix} x'_1 \\ x'_2 \end{pmatrix} = \begin{pmatrix} \cos m\theta_k & -\sin m\theta_k \\ \sin m\theta_k & \cos m\theta_k \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}.$$

2.2 Why the Score Becomes Relative

Let R_m denote the block-diagonal rotation for position m . RoPE applies R_m to the query at position m and R_n to the key at position n . Their dot product is

$$(R_m \mathbf{q})^\top (R_n \mathbf{k}) = \mathbf{q}^\top R_m^\top R_n \mathbf{k} = \mathbf{q}^\top R_{n-m} \mathbf{k},$$

using the property of rotation matrices that $R_m^\top R_n = R_{n-m}$. The score depends on $\mathbf{q}$, $\mathbf{k}$, and the *difference* $n - m$ only — never on absolute positions. This is the geometric reason RoPE generalizes across positions.

Context-extension methods (positional interpolation, YaRN) rescale the frequencies θ_k so a model trained at one length runs at a longer one [B3: Ch. 1]. Because the score depends on $(n - m)\theta_k$, shrinking θ_k is equivalent to compressing the effective distance — which is why these methods extend context without retraining from scratch.

LoRA: Low-Rank Adaptation

[B3: Ch. 4] Full fine-tuning updates a weight matrix $W_0 \in \mathbb{R}^{d \times k}$ to $W_0 + \Delta W$, learning all $d \times k$ entries of ΔW . LoRA constrains ΔW to be low rank.

3.1 The Factorization

LoRA writes the update as a product of two thin matrices:

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k).$$

The adapted forward pass is

$$h = W_0 x + \Delta W x = W_0 x + B(Ax),$$

with W_0 frozen and only A, B trained. In practice the update is scaled by α/r for a constant α , so the effective learning rate is decoupled from the chosen rank.

3.2 The Parameter Saving

Full fine-tuning trains $d \times k$ parameters per matrix. LoRA trains $r(d + k)$. For $d = k = 4096$ and $r = 8$:

$$\frac{r(d + k)}{dk} = \frac{8 \times 8192}{4096 \times 4096} = \frac{65,536}{16,777,216} \approx 0.0039,$$

a $256\times$ reduction in trainable parameters for that matrix. Because gradients and optimizer state are stored only for A and B , the memory saving (recall the optimizer-state multiplier of [B2: Ch. 4]) is of the same order — the mechanism behind fitting fine-tuning onto a single GPU.

The premise is that the *update* needed to adapt a pretrained model has low intrinsic rank, even though the model itself is full rank. QLoRA combines this with a 4-bit frozen base [B2: Ch. 7]: the large matrix W_0 is quantized, while the small A, B train in higher precision.

Alignment Objectives

4.1 PPO for RLHF

[B3: Ch. 5] RLHF fine-tunes a policy π_θ (the language model) to maximize a learned reward r_ϕ while staying close to a reference policy π_{ref} (the SFT model). The objective is

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [r_\phi(x, y)] - \beta \text{KL}(\pi_\theta(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)).$$

The reward pulls the policy toward human-preferred outputs; the KL penalty, weighted by β , prevents it from drifting so far that it produces degenerate text that games the reward model. PPO optimizes this with a clipped surrogate objective that limits how much the policy changes per update, for stability.

4.2 The Reward Model

r_ϕ is trained from pairwise human preferences. Given a prompt x with a preferred response y_w and a rejected one y_l , the model maximizes the probability that it scores the winner higher, under the Bradley–Terry model:

$$\mathcal{L}(\phi) = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))],$$

where σ is the logistic function.

4.3 DPO: Skipping the Reward Model

[B3: Ch. 5] Direct Preference Optimization shows the RLHF objective has a closed-form optimal policy, which lets one optimize the preference data *directly* on the policy, with no separate reward model and no RL loop:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right].$$

DPO’s loss is the reward-model loss with the reward replaced by the policy’s own log-ratio against the reference. It is the same preference signal, routed directly into the policy — which is why it is simpler and more stable than PPO-based RLHF, at comparable quality.

Distributed Communication

[B2: Ch. 5] This appendix gives the communication cost that the distributed-training chapter describes qualitatively.

5.1 All-Reduce

Data-parallel training averages gradients across N workers with an all-reduce. The efficient ring all-reduce moves each worker’s data around a ring in two phases (reduce-scatter, then all-gather). For a gradient buffer of size S bytes, each worker sends and receives approximately

$$2 \cdot \frac{N-1}{N} \cdot S \text{ bytes,}$$

which is nearly independent of N for large N — the property that makes data parallelism scale. The cost is paid every step, which is why the interconnect (NVLink within a node, InfiniBand across nodes, [B2: Ch. 1]) determines feasibility.

5.2 FSDP Communication

[B2: Ch. 5] Fully Sharded Data Parallelism shards parameters across workers and reconstructs each layer’s full parameters on demand with an all-gather, then frees them after use. Per training step this adds, relative to plain data parallelism:

- an **all-gather** of parameters in the forward pass,
- an **all-gather** again in the backward pass,
- a **reduce-scatter** of gradients (replacing the all-reduce).

The trade is explicit: FSDP cuts per-worker memory from $O(\text{full model})$ to $O(\text{model}/N)$ at the cost of roughly $1.5\times$ the communication volume of plain data parallelism. This is the memory-versus-communication dial that the trilogy’s closing theme names — communication, like bandwidth, is the true constraint, not raw compute.

5.3 Expert Parallelism (MoE)

[B1: Ch. 5], [B2: Ch. 5] Mixture-of-Experts routing sends each token to its chosen experts, which live on different workers. This requires an **all-to-all** exchange: every worker sends its tokens to the workers holding their target experts, then receives the processed results back.

All-to-all is the most communication-intensive primitive here, and load imbalance (a few popular experts) makes it worse — the reason MoE training adds a load-balancing loss.

Part II

Annotated Bibliography

Foundational References

Each entry names the work, its authors and year, and one line on why it matters and where the trilogy uses it. This is a study bibliography, not an exhaustive survey: it lists the papers a reader should go to first for each concept.

6.1 Transformers and Attention

- **Attention Is All You Need** — Vaswani et al., 2017. Introduced the transformer and scaled dot-product attention; the architecture the entire trilogy rests on. [B1: Ch. 1]
- **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness** — Dao et al., 2022. The IO-aware, tiled attention kernel with online softmax; reframed attention as a memory-movement problem. [B1: Ch. 3], [B2: Ch. 9]
- **FlashAttention-2** — Dao, 2023. Improved parallelism and work partitioning over the original; the version in most current frameworks. [B2: Ch. 9]
- **GQA: Training Generalized Multi-Query Transformer Models** — Ainslie et al., 2023. Grouped-query attention, sharing K/V across query heads to shrink the KV cache. [B1: Ch. 2]

6.2 Positional Encoding

- **RoFormer: Enhanced Transformer with Rotary Position Embedding** — Su et al., 2021. Introduced RoPE, the rotation-based relative encoding used by Llama, Mistral, and Qwen. [B3: Ch. 1]
- **YaRN: Efficient Context Window Extension of Large Language Models** — Peng et al., 2023. Frequency rescaling to extend a model's context beyond its training length. [B3: Ch. 1]

6.3 Architectures Beyond the Transformer

- **Transformer-XL** — Dai et al., 2019. Segment-level recurrence to extend attention beyond a fixed window. [B3: Ch. 2]
- **Efficiently Modeling Long Sequences with Structured State Spaces (S4)** — Gu et al., 2021. The state-space sequence model; near-linear cost on long sequences. [B3: Ch. 2]

- **Mamba: Linear-Time Sequence Modeling with Selective State Spaces** — Gu and Dao, 2023. Made state-space models input-dependent and competitive with transformers. [B3: Ch. 2]
- **Griffin: Mixing Gated Linear Recurrences with Local Attention** — De et al., 2024. A hybrid recurrence/attention architecture for efficient long-context inference. [B3: Ch. 2]
- **Perceiver: General Perception with Iterative Attention** — Jaegle et al., 2021. Cross-attends large inputs into a small latent set, decoupling compute from input size. [B3: Ch. 2]

6.4 Scaling and Training

- **Scaling Laws for Neural Language Models** — Kaplan et al., 2020. Established the power-law relationship between loss, parameters, data, and compute. [B3: Ch. 3]
- **Training Compute-Optimal Large Language Models (Chinchilla)** — Hoffmann et al., 2022. Showed most large models were undertrained; the ~ 20 -tokens-per-parameter rule. [B3: Ch. 3]
- **Mixed Precision Training** — Micikevicius et al., 2017. FP16 training with loss scaling and an FP32 master copy. [B2: Ch. 4], [B3: Ch. 6]

6.5 Fine-Tuning and Alignment

- **LoRA: Low-Rank Adaptation of Large Language Models** — Hu et al., 2021. Low-rank weight updates for parameter-efficient fine-tuning. [B3: Ch. 4]
- **QLoRA: Efficient Finetuning of Quantized LLMs** — Dettmers et al., 2023. LoRA on a 4-bit frozen base; fine-tuning a 65B model on one GPU. [B3: Ch. 4]
- **Training Language Models to Follow Instructions with Human Feedback (InstructGPT)** — Ouyang et al., 2022. The canonical RLHF pipeline: SFT, reward model, PPO. [B3: Ch. 5]
- **Direct Preference Optimization** — Rafailov et al., 2023. Reframed RLHF as a direct loss on preferences, removing the reward model and RL loop. [B3: Ch. 5]
- **Proximal Policy Optimization Algorithms** — Schulman et al., 2017. The clipped-surrogate RL algorithm used in classical RLHF. [B3: Ch. 5]

6.6 Mixture of Experts

- **Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer** — Shazeer et al., 2017. Introduced sparse expert routing with a gating network. [B1: Ch. 5]
- **Switch Transformers** — Fedus et al., 2021. Simplified MoE routing to top-1 and scaled it; popularized active-vs-total parameters. [B1: Ch. 5], [B2: Ch. 5]

6.7 Inference Systems and Serving

- **Efficient Memory Management for LLM Serving with PagedAttention (vLLM)** — Kwon et al., 2023. Paged KV-cache memory; the basis of modern high-throughput serving. [B1: Ch. 3], [B2: Ch. 6]

- **Orca: A Distributed Serving System for Transformer-Based Generative Models** — Yu et al., 2022. Introduced continuous (iteration-level) batching. [B1: Ch. 3]
- **SGLang: Efficient Execution of Structured Language Model Programs** — Zheng et al., 2023. RadixAttention for automatic prefix-cache reuse across requests. [B1: Ch. 5], [B2: Ch. 6]
- **Fast Inference from Transformers via Speculative Decoding** — Leviathan et al., 2022. Draft-and-verify decoding with provably identical output distribution. [B1: Ch. 3], [B2: Ch. 7]

6.8 Quantization

- **LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale** — Dettmers et al., 2022. 8-bit inference with outlier handling. [B2: Ch. 7]
- **GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers** — Frantar et al., 2022. Accurate one-shot weight quantization. [B2: Ch. 7]
- **AWQ: Activation-Aware Weight Quantization** — Lin et al., 2023. Protects salient (outlier) weights; the basis for the reasoning-vs-bit-width caveat. [B2: Ch. 7]

6.9 Memory-Saving Training

- **Training Deep Nets with Sublinear Memory Cost** — Chen et al., 2016. Gradient checkpointing: recompute activations to save memory. [B3: Ch. 6]
- **ZeRO: Memory Optimizations Toward Training Trillion Parameter Models** — Rajbhandari et al., 2019. The sharding strategy behind FSDP. [B2: Ch. 5]
- **Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism** — Shoeybi et al., 2019. Tensor parallelism for transformers. [B2: Ch. 5]
- **GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism** — Huang et al., 2018. Pipeline parallelism with micro-batching to reduce bubbles. [B2: Ch. 5]

6.10 Retrieval and Reasoning

- **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks** — Lewis et al., 2020. The original RAG formulation. [B1: Ch. 8, 9]
- **Lost in the Middle: How Language Models Use Long Contexts** — Liu et al., 2023. Documented the middle-of-context attention sag; the basis for retrieval ordering. [B1: Ch. 9]
- **Chain-of-Thought Prompting Elicits Reasoning in Large Language Models** — Wei et al., 2022. Intermediate reasoning steps improve hard-problem accuracy. [B1: Ch. 12]
- **Self-Consistency Improves Chain of Thought Reasoning** — Wang et al., 2022. Sample multiple paths and vote. [B1: Ch. 12]

For publication, each in-text concept in the three books should carry a citation to its entry here. The grouping above mirrors the trilogy's structure, so attaching references chapter by chapter is mechanical: attention and serving papers map to Books 1–2, architecture and training papers to Book 3.

Part III

Glossary

Cross-Book Glossary

Terms that recur across the trilogy, with a concise definition and a pointer to where each is introduced. Entries are alphabetical.

Active parameters

[B1: Ch. 5]

In a Mixture-of-Experts model, the subset of parameters a single token actually passes through. Determines per-token compute and decode speed, as opposed to total parameters, which determine memory.

All-reduce

[B2: Ch. 5]

The collective operation that averages gradients across data-parallel workers each training step. The ring implementation makes its per-worker cost nearly independent of worker count.

Arithmetic intensity

[B2: Ch. 1]

FLOPs performed per byte read from memory. High for prefill (compute-bound), very low for decode (memory-bandwidth-bound). The axis of the roofline model.

Attention

[B1: Ch. 1]

The operation by which each token gathers information from others via query-key-value matching. The core of the transformer.

BF16 (bfloat16)

[B2: Ch. 4], [B3: Ch. 6]

A 16-bit float keeping FP32's 8-bit exponent (wide range) at the cost of mantissa precision. The default for large-model training because it rarely overflows.

Causal attention

[B3: Ch. 2]

Self-attention masked so a token attends only to itself and earlier tokens. What makes a decoder generate left-to-right.

Continuous batching

[B1: Ch. 3]

Serving technique that admits and evicts requests from the running batch at every decode step, keeping the GPU near full utilization.

Cross attention

[B3: Ch. 2]

Attention where queries come from one sequence and keys/values from another. The bridge in encoder-decoder and multimodal models.

Decode

[B1: Ch. 1]

The autoregressive phase: one forward pass per output token, strictly sequential, memory-bandwidth-bound.

DPO (Direct Preference Optimization)

[B3: Ch. 5]

Alignment method that optimizes preference data directly on the policy, removing RLHF's separate reward model and RL loop.

Flash Attention

[B1: Ch. 3], [B2: Ch. 9]

An IO-aware attention kernel that tiles the computation in SRAM and uses online softmax, turning attention's HBM traffic from $O(N^2)$ to $O(N)$.

FSDP (Fully Sharded Data Parallelism)

[B2: Ch. 5]

Distributed strategy that shards weights, gradients, and optimizer state across workers, gathering each layer's parameters only when needed.

GQA (Grouped-Query Attention)

[B1: Ch. 2]

Sharing key/value heads across groups of query heads to shrink the KV cache with minimal quality loss.

Gradient accumulation

[B3: Ch. 6]

Summing gradients over several micro-batches before updating, simulating a large batch with small-batch memory.

Gradient checkpointing

[B3: Ch. 6]

Saving only some activations in the forward pass and recomputing the rest during backprop, trading compute for activation memory.

HBM (High-Bandwidth Memory)

[B2: Ch. 1]

The large but comparatively slow on-package GPU DRAM where weights and the KV cache live. Its bandwidth is the decode bottleneck.

KV cache

[B1: Ch. 1, 2]

Stored key and value tensors from prefill, reused during decode so past tokens are never recomputed. Grows one row per generated token.

LoRA (Low-Rank Adaptation)

[B3: Ch. 4]

Parameter-efficient fine-tuning that freezes the base weights and learns a small low-rank update BA .

Mixed precision

[B2: Ch. 4]

Training in 16-bit for speed while keeping a 32-bit master copy of weights for stability.

MoE (Mixture of Experts)

[B1: Ch. 5]

An architecture replacing the FFN with many expert FFNs and a router that activates only a few per token. High total capacity, low active compute.

Online softmax

[B2: Ch. 9]

Computing softmax incrementally with a running maximum and sum, rescaling as new tiles appear. The trick that lets Flash Attention avoid materializing the score matrix.

PagedAttention

[B1: Ch. 3], [B2: Ch. 6]

KV-cache memory management in fixed non-contiguous blocks with a per-request block table, eliminating fragmentation. The basis of vLLM.

Prefill

[B1: Ch. 1]

The phase that processes all input tokens in parallel in one forward pass, builds the KV cache, and emits the first token. Compute-bound; sets TTFT.

Quantization

[B1: Ch. 3], [B2: Ch. 7]

Representing weights or KV cache in lower precision (INT8/INT4) to cut memory and bandwidth, with care for outlier weights.

RAG (Retrieval-Augmented Generation)

[B1: Ch. 8, 9]

Retrieving relevant documents and inserting them into the prompt so the model answers from provided context.

RLHF (Reinforcement Learning from Human Feedback)

[B3: Ch. 5]

Aligning a model by training a reward model on human preferences, then RL-optimizing the policy against it under a KL constraint.

RoPE (Rotary Position Embedding)

[B3: Ch. 1]

Encoding position by rotating query/key vectors, so attention scores depend only on relative position. The modern default.

Roofline model

[B2: Ch. 1]

A plot bounding achievable performance by compute and memory-bandwidth ceilings against arithmetic intensity. Shows why decode is memory-bound.

Scaling laws

[B3: Ch. 3]

The empirical power-law relating model loss to parameters, data, and compute; the basis for compute-optimal sizing.

Speculative decoding

[B1: Ch. 3], [B2: Ch. 7]

A small draft model proposes tokens that the target model verifies in parallel, accelerating decode without changing the output distribution.

State-space model (S4, Mamba)

[B3: Ch. 2]

A sequence architecture using a structured linear recurrence; near-linear inference cost while remaining parallelizable in training.

Temperature

[B1: Ch. 5]

A sampling parameter dividing logits before softmax; low values sharpen the distribution (deterministic), high values flatten it (diverse).

Tensor parallelism

[B2: Ch. 5]

Splitting each layer's weight matrices across GPUs that cooperate on every token. Needs fast interconnect, so it stays within a node.

Tiling

[B2: Ch. 8]

Loading small blocks of data into fast SRAM and reusing them many times, raising arithmetic intensity. The core kernel-optimization pattern.

Tokenisation

[B3: Ch. 3]

Converting text into integer token IDs from a fixed vocabulary, typically via byte-pair encoding.

top_p (nucleus sampling)

[B1: Ch. 5]

Sampling from the smallest set of tokens whose cumulative probability exceeds p ; adapts to the distribution's shape.

TTFT (Time To First Token)

[B1: Ch. 1], [B2: Ch. 6]

Latency from request to first generated token. Dominated by prefill once the request reaches the model; includes queue delay in production.

How the Companion Maps to the Trilogy

Companion part	Serves
Mathematical Appendices	readers wanting the algebra behind a stated result; reviewers checking rigor
Annotated Bibliography Glossary	attribution for every major concept; a reading path to go deeper quick lookup of any recurring term, with a pointer back to its home chapter

Together with the three main books, this companion completes the editorial layer a reviewer asked for: the trilogy teaches the ideas with diagrams and intuition, and the companion supplies the derivations, the citations, and the definitions in one reference shelf.